

Algorithmic Simplification of Neural Networks

Mosaic-of-Motifs (MoMos)

Pedram Bakhtiarifard

Department of Computer Science
University of Copenhagen, ML Section

pba@di.ku.dk

pedrambakh.com

<https://sainthoodlab.github.io>

September 2, 2026

Overview

1. Intuition

- 1.1 Learning is compression
- 1.2 Analogies from real life

2. Algorithmic Simplification

- 2.1 A model learns, describe its structure
- 2.2 How does MoMos parameterization look?

3. Insights

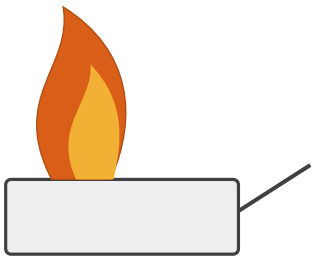
- 3.1 Ablation on capacity vs. performance
- 3.2 Optimization domain and more ...

4. Takeaways

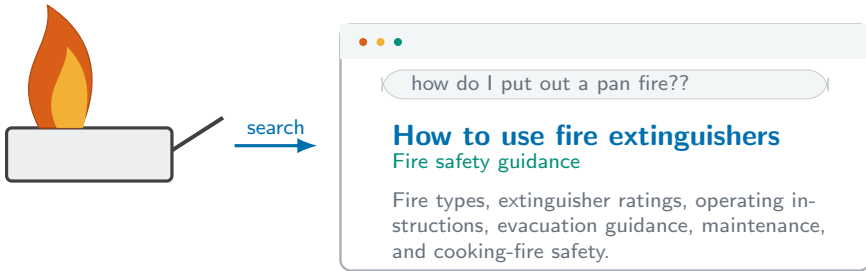
Learning is compression

"it is not unreasonable to say that when you a presented a lot of information, if you are able to parse the information and make use of it against your task, more often than not, you have compressed the information."

– Overheard at ATDL, Sep. 3rd 2026



A pan catches fire.



Using a fire extinguisher

Fire safety guidance

Portable extinguishers use different agents and carry ratings showing which fires they can fight. They should remain visible, accessible, charged, and in working condition. Before approaching a fire, alert others, make sure help is being called, choose an extinguisher rated for the material burning, and keep a clear escape route. Use it only on a small, contained fire. Pull the pin, aim at the base, squeeze the handle, and sweep from side to side. If the extinguisher empties, the fire grows, or smoke threatens your exit, evacuate immediately. For a small grease fire in a pan, turn off the burner, slide a lid or baking sheet over the pan, do not move it, and leave it covered until cool.

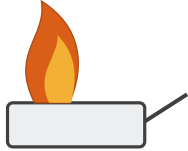
Paraphrased from OSHA and U.S. Fire Administration guidance.

Using a fire extinguisher

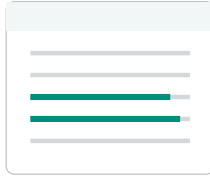
Fire safety guidance

Portable extinguishers use different agents and carry ratings showing which fires they can fight. They should remain visible, accessible, charged, and in working condition. Before approaching a fire, **alert others and make sure help is being called**, choose an extinguisher **rated for the material burning**, and keep a **clear escape route**. Use it only on a **small, contained fire**. **Pull the pin, aim at the base, squeeze the handle, and sweep from side to side**. If the extinguisher empties, the fire grows, or smoke threatens your exit, **evacuate immediately**. For a small grease fire in a pan, **turn off the burner, slide a lid or baking sheet over the pan, do not move it, and leave it covered until cool**.

Paraphrased from OSHA and U.S. Fire Administration guidance.



search
→



compress
→

Useful knowledge

- 1 Heat off → cover pan
- 2 Pull → aim at base
Squeeze → sweep
- 3 Escape if unsafe



Found **structure**, kept necessary, and discards the rest.

Analogies from real life

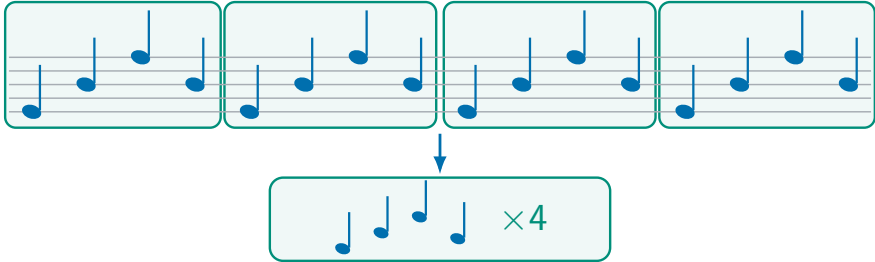
Sixteen separate notes



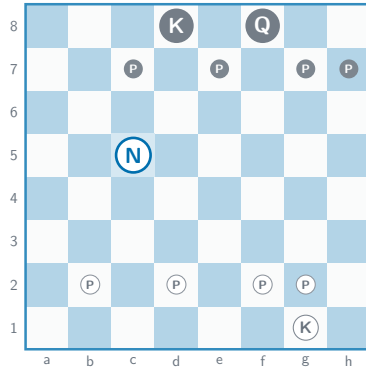
The same four-note motif

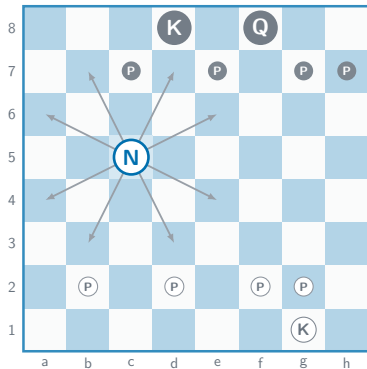


The same four-note motif

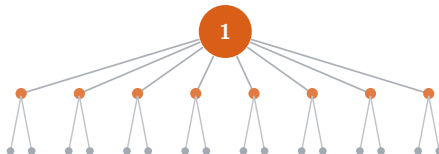


Remember one pattern and its repeat count

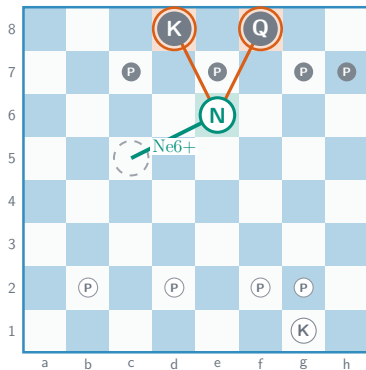




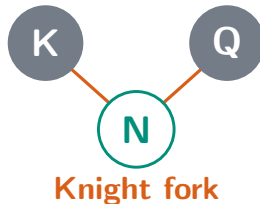
Where should the knight move?



each move creates more lines; memorize?



One useful pattern

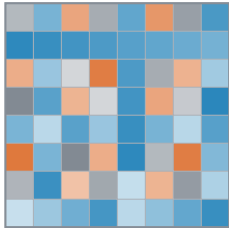


Remember and compute Ne6+ lines

Well, what about deep learning?

Individual weights or reusable structure?

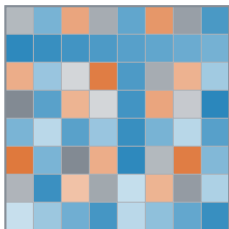
$$W \in \mathbb{R}^n$$



random initialization

A trivial description of every weight.

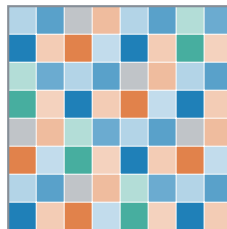
$$W \in \mathbb{R}^n$$



random initialization

A trivial description of every weight.

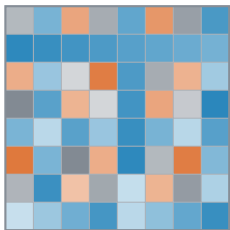
training



after training

Induced structure

$$W \in \mathbb{R}^n$$



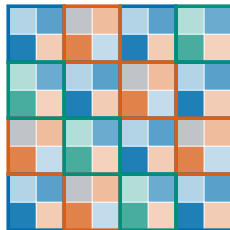
random initialization

A trivial description of every weight.

training



$$W \in \mathbb{R}^{k \times s}$$

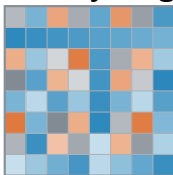


after training

Induced structure

Kolmogorov complexity of the weights

List every weight

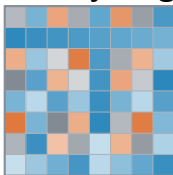


n weights, q bits per weight

$$\mathcal{K}(W) \leq nq + \mathcal{O}(1)$$

Kolmogorov complexity of the weights

List every weight

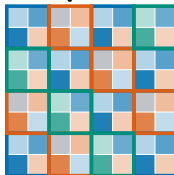


n weights, q bits per weight

reuse



List patterns



$A \times 7$

$B \times 5$

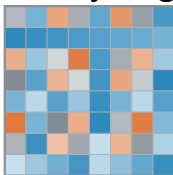
$C \times 4$

$$\mathcal{K}(W) \leq nq + \mathcal{O}(1)$$

$$\mathcal{K}(W^*) \leq nq - q \sum_j p_j$$

where $p_j = s(r_j - 1)$ and $r_j \in \mathbb{N}_{\geq 1}$ is the number of occurrences of pattern j .

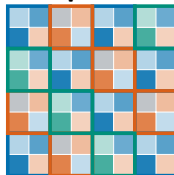
List every weight



reuse



List patterns



$$nq = 64q$$

$n = 64$ weights, each stored with q bits

$$64q - q(24 + 16 + 12) = 12q$$

$$p_A = 4(7 - 1) = 24,$$

$$p_B = 4(5 - 1) = 16,$$

$$p_C = 4(4 - 1) = 12$$

**Okay, but we assumed that
the structure appeared.**

*And we need to know where patterns are located

Algorithmic Simplification of Neural Networks with Mosaic-of-Motifs

Pedram Bakhtiarifard, Tong Chen, Jonathan Wenshøj,
Erik B. Dam, Raghavendra Selvan

Department of Computer Science, University of Copenhagen, Denmark

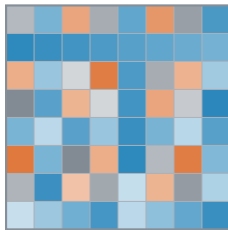
Correspondence: {pba,raghav}@di.ku.dk

Date: April 2026 * Code: github.com/saintslab/MoMos

Abstract

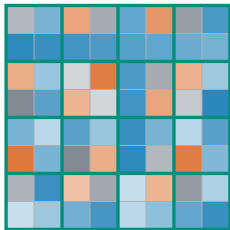
Large-scale deep learning models are well-suited for compression. Across a variety of tasks, methods like pruning, quantization, and knowledge distillation have been used to achieve massive reductions in model parameters with only marginal performance drops. This raises the central question: *Why are deep neural networks suited for compression?* In this work, we take up the perspective of algorithmic complexity to explain this behavior. We hypothesize that the parameters of trained models have more structure and, hence, exhibit lower algorithmic complexity compared to the weights at (random) initialization. Furthermore, model compression methods harness this reduced algorithmic complexity to compress models. Although an unconstrained parameterization of model weights, $\mathbf{w} \in \mathbb{R}^n$, can represent arbitrary weight assignments, the solutions found during training exhibit repeatability and structure, making them simpler to implement than a trivial program. To this end, we formalize the Kolmogorov complexity of $\mathbf{w}$ by $\mathcal{K}(\mathbf{w})$. We introduce a constrained parameterization $\hat{\mathbf{w}}$ that partitions parameters into blocks of size s and restricts each block to be selected from a set of k reusable motifs, specified by a reuse pattern (or mosaic). The resulting method, *Mosaic-of-Motifs* (MoMos), provides a theoretically justified parameterization that biases optimization toward algorithmically simpler solutions. Empirical evidence from multiple experiments shows that MoMos consistently lowers the algorithmic complexity of neural networks during training while preserving the performance of unconstrained models. These results suggest that parameter compressibility is not only observed after training, but can be induced from the optimization domain.

Start from an ordinary optimizer update



$$W^{(t)} \in \mathbb{R}^n$$

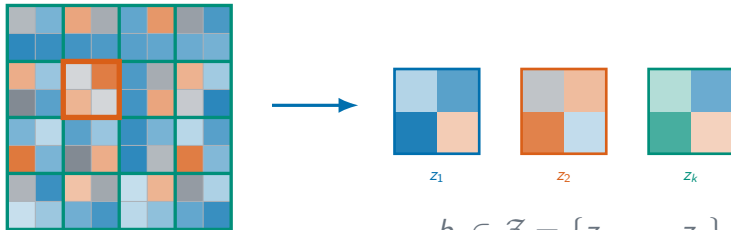
Partition the weights into m blocks of size s



$$\psi(W^{(t)}) = (b_1, \dots, b_m), \quad b_i \in \mathbb{R}^s$$

$m = \lceil n/s \rceil$ and each 2×2 block has $s = 4$.

Each block must choose one of only k motifs



$$b_i \in \mathcal{Z} = \{z_1, \dots, z_k\}$$

m block positions, but at most k distinct block values.

Store only k reusable motifs

Motif bank $\mathcal{Z}$



k reusable blocks

The mosaic says which motif each block uses

Motif bank $\mathcal{Z}$



k reusable blocks

Mosaic $\mathcal{M}$



A	B	A	C
C	A	B	A
B	C	A	B
A	B	C	A

one index per block

Motifs and mosaic together describes weights

Motif bank $\mathcal{Z}$



z_1



z_2



z_3

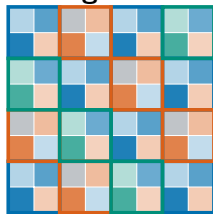
k reusable blocks

Mosaic $\mathcal{M}$



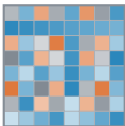
one index per block

Weights W^*



$$\mathcal{K}(W^*) \leq ksq + m \lceil \log_2 k \rceil + O(1)$$

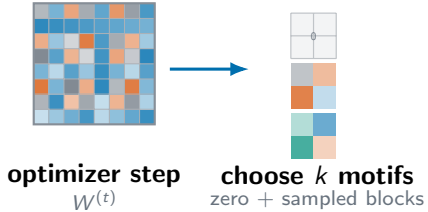
The optimizer proposes an ordinary update



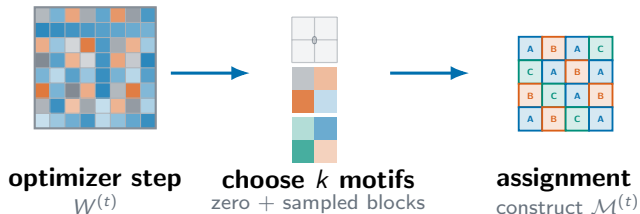
optimizer step

$W^{(t)}$

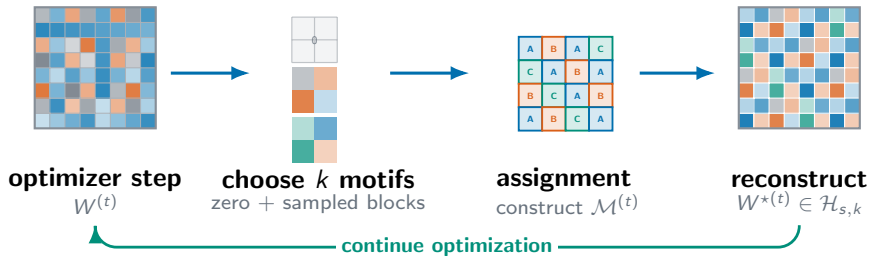
Choose a small motif bank



Each block selects its nearest motif



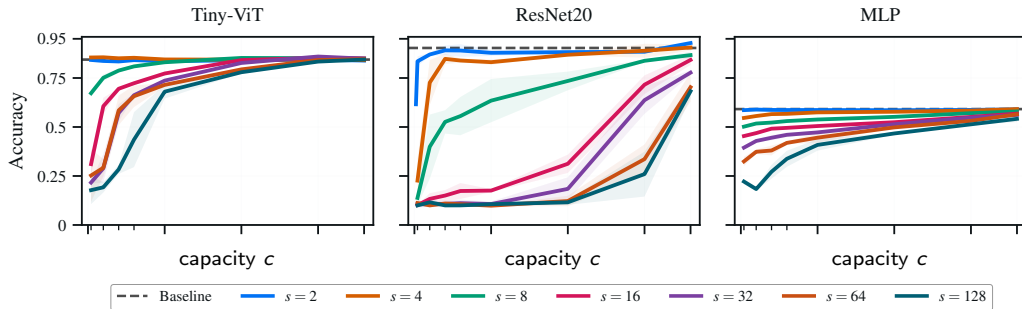
Project the update back into motif reuse



Does it work?

*Some insights

Small blocks retain performance under strong reuse.



At $s = 2$ and $c = 5\%$, near-baseline performance comes with:

Tiny-ViT

3.02×

algorithmic simplification

ResNet20

3.95×

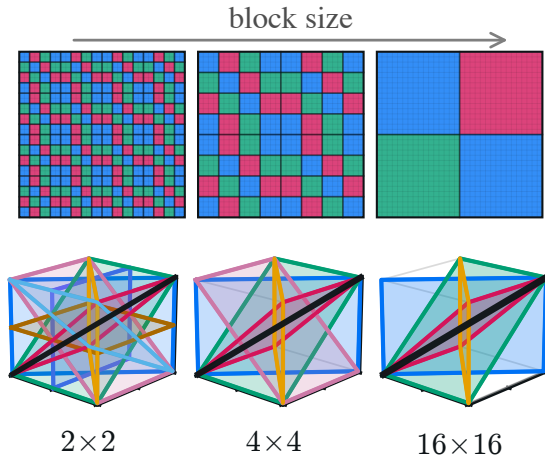
algorithmic simplification

MLP

3.52×

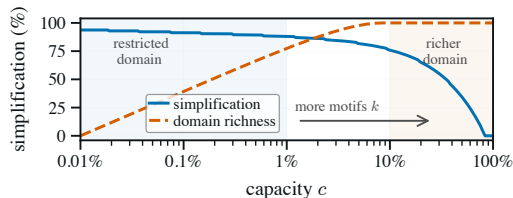
algorithmic simplification

Smaller blocks preserve a richer optimization domain.



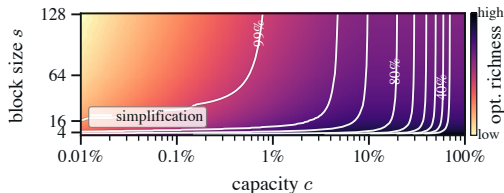
smaller blocks $\Rightarrow$ **more arrangements** $\Rightarrow$ **richer domain**

Capacity and block size control the trade-off.



Capacity $c = k/m$

More motifs enrich the domain,
but lengthen the description.

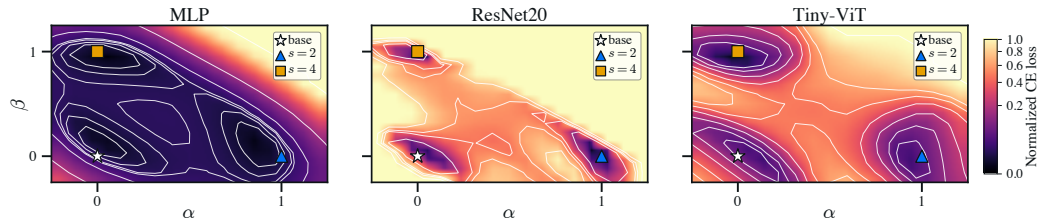


Block size s

Smaller blocks create more arrangements,
enriching the domain at a fixed budget.

Use small motifs at the lowest capacity that task the task.

Different weights, similar decision geometry.



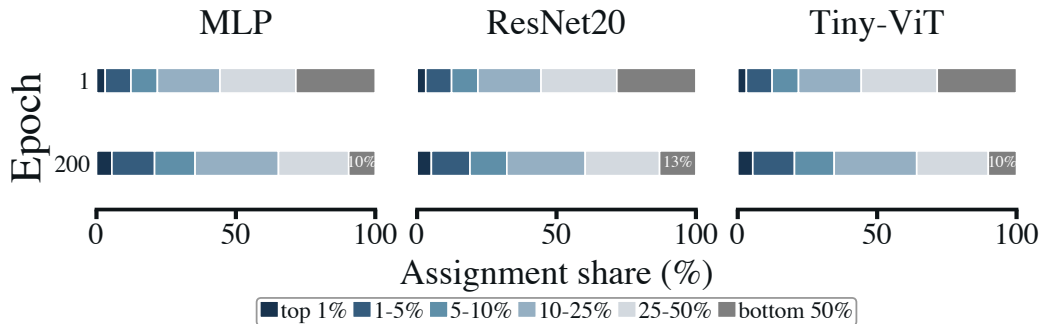
Star: dense baseline.

Triangle: $s = 2$.

Square: $s = 4$.

MoMos steers learning toward a **different structured parameterization**.

Training concentrates reuse beyond the worst-case bound.



motifs concentrate $\Rightarrow$ the bound is conservative

Takeaways

I hope I have (partially) convinced you that...

- **learning goes hand in hand with compression**
- **complexity does not equate scale**
- **structure can be optimized directly**

Thank you.

Pedram Bakhtiarifard

PhD Candidate in Sustainable Machine Learning
SAINTS Lab, University of Copenhagen

pba@di.ku.dk

pedrambakh.com

saintslab.github.io



DATAPACT



INDEPENDENT
RESEARCH FUND
DENMARK

